

深圳大学实验报告

课程名称： 算法设计与分析

实验项目名称： 回溯法——地图填色问题

学院： 计算机与软件学院

专业： 计算机科学与技术

指导教师： 刘刚

报告人： 汉雨 学号： 2024040042

实验时间： 2026年4月 日—2026年5月13日

实验报告提交时间： 2026年5月14日

教务部制

一、实验目的：

- 1、掌握回溯法算法设计思想。
- 2、掌握地图填色问题的回溯法解法。

二、实验内容：

- 1、平面图与四色定理
- 2、小规模数据，利用四色填色测试算法的正确性
- 3、针对三个地图数据尝试分别使用 5 个(1e450_5a), 15 个(1e450_15b), 25 个(1e450_25a) 颜色为地图着色
- 4、随机产生不同规模的**平面图（四色定理）**，分析算法效率与图规模的关系。

三、实验内容及过程：

平面图和四色定理

1.1 平面图

首先图论中的图由“点”和“边”组成。而平面图是指边与边之间除了端点之外不能相交的图。

补充：我之前有误区---如果为什么不能把中间相交的线设为点 E，这样不就没有相交线了吗。但其实图中的点是一开始就规定好的，并不是画成相交的样子就是“交点”。

1.2 四色定理

一张平面地图，若要使地图的某一个国家与相邻国家的颜色都不相同至少要用几种颜色？

答：四种颜色。



图 1-1

2. 小规模数据验证四色定理

现有如下小规模地图数据。我们可以将其抽象成下方右图中的平面图示意图：

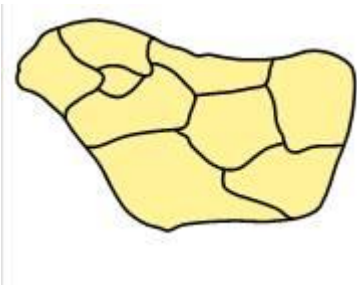


图 1-2

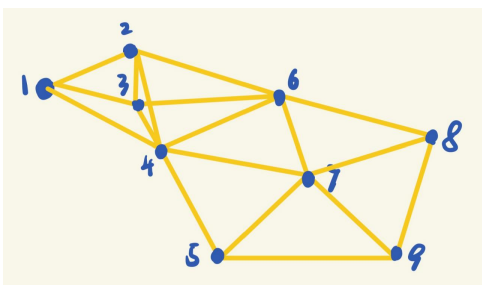


图 1-3

可以仿照实验附件，将图片信息整理成文件名为“small_map.col”的“.col”文件：

```
1 c FILE: small_map.col
2 c DESCRIPTION: Small planar map for testing four-coloring algorithm
3 c number of vertices : 10
4 c number of edges : 19
5 c number of colors : 4
6 p edge 10 19
7 e 1 2
8 e 1 5
9 e 1 8
```

图 1-5

根据我们设计的算法，可得此次实验的伪代码：

Algorithm: FourColoringBacktracking

Input:

file: graph data file in .col format
colorNum = 4

Output:

A valid coloring scheme color[], or report failure

1. Read graph data from file
2. Build the adjacency list graph
3. Initialize color[] to 0 for all vertices
4. Start the timer
5. If Backtrack(1) returns true:
 - Output the color assigned to each vertex
 - If CheckResult() returns true:
 - Output "Validation successful"
 - Else:
 - Output "Validation failed"
- Else:
 - Output "No valid four-coloring solution found"
6. Stop the timer
7. Output the running time

实验结果运行时间 **0.043876 ms**，根据实验结果得出的填色示意图如下：

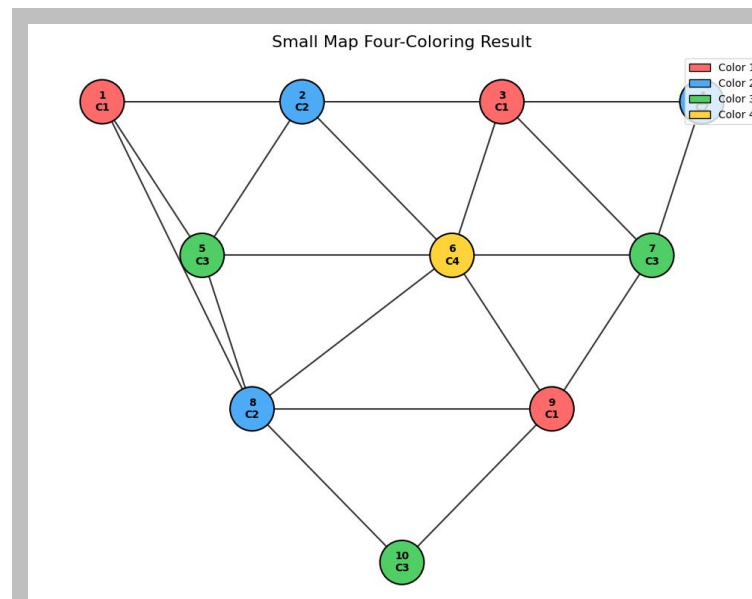


图 1-6

回溯法及其优化实现地图填色

原始回溯法实现地图填色

从第一个节点开始，给每个节点用 for 循环依次尝试所有颜色；如果当前颜色和相邻节点不冲突，就继续染下一个节点，若没有下一个点了则输出解，若颜色与周围区域产生冲突则换下一个颜色尝试；若对于一个节点的所有颜色都尝试完了，则返回上一个点；若已经返回到第 0 个点，都没能找到解，则说明该题无解。

以上文字可以用流程图表示：

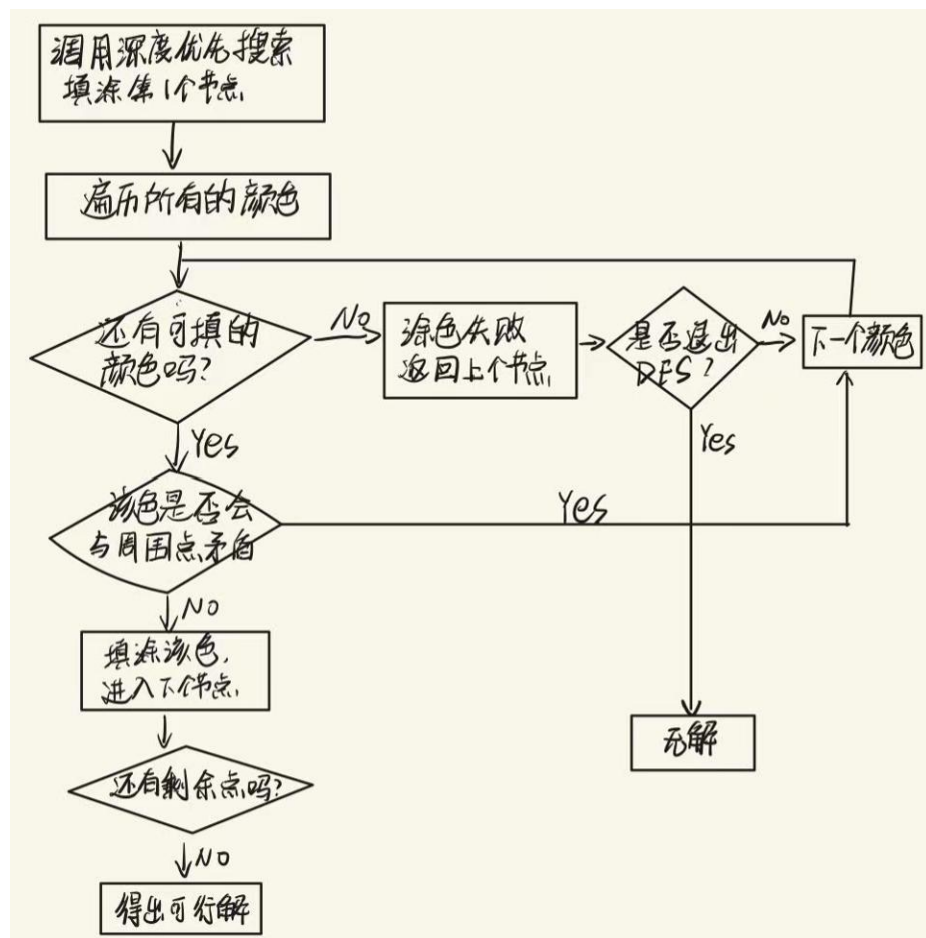


图 2-1

关键部分的伪代码实现：

```

bool dfs(int x) {
    if (x == n) {
        return true; // 所有节点都染完，找到可行解
    }

    for (int c = 1; c <= m; c++) {
        if (当前节点 x 染颜色 c 不冲突) {
            color[x] = c;

```

```

        if (dfs(x + 1)) {
            return true;
        }

        color[x] = 0; // 撤销, 回溯
    }
}

return false; // 所有颜色都试完, 当前节点无法染色
}

```

对与“回溯法”来说，一个重要的优化方向就是“剪枝”，提前剪去一些不可能成立的解。可以通过以下几个角度进行优化：

最少剩余值优先：

我们可以想想自己是怎么对地图进行填色的？我们一般会对“条件较为苛刻”的节点（也就是剩余可填色数目较少的节点）优先填色。

因此可以对节点类再设置一个属性：表示这个节点还剩下多少个颜色可以涂。

随后在下一次对节点填色之前，可以先检查“可涂颜色”个数这个属性，在所有的点中找到剩余可填涂颜色最少的点，遍历该点剩余可以填涂的颜色进行搜索，并在涂完该点之后，需要对该点的临接点的两个属性——剩余可填涂颜色以及剩余可填涂颜色的数量，进行更新。要注意的是，在回溯时，需要对临接点的修改进行复原。

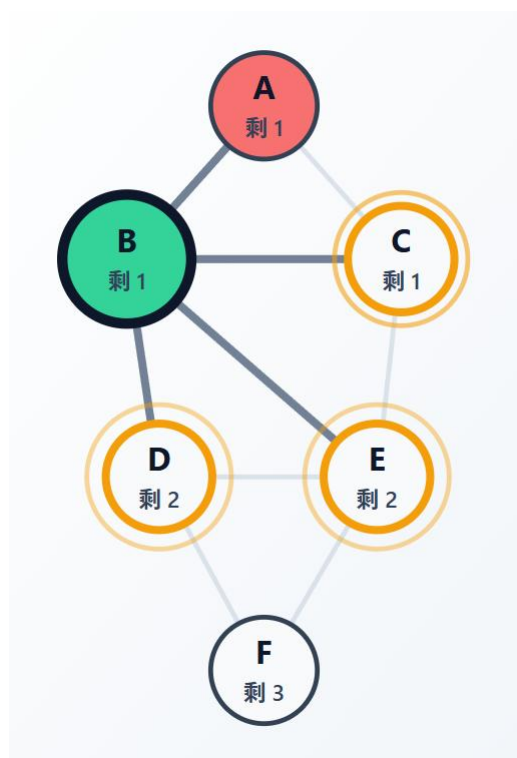


图 2-2

这个方法为什么可以称为“剪枝”？如果我们对点 n 进行涂色，涂色完发现另一个点 m 的“可涂颜色数”变成了 0，这时就代表无解，我们没必要浪费时间吧这个解进行进一步的扩充求解了。

伪代码如下：

```
bool dfs(int step) {
    if (step > 点数) return true;

    选出一个还没染色且可选颜色最少的点 u;

    枚举 u 可以使用的颜色 c {
        给 u 染成 c;
        更新相邻点的可选颜色状态;

        if (当前状态合法 && dfs(step + 1))
            return true;

        撤销本次染色和状态更新;
    }

    return false;
}
```

最少剩余值优先+度数大的点优先:

我们不仅可以对剩余值讨论，还可以引入图论里的“度”。

度是指：无向图中，与某点相邻的点的数量。在我们的问题中，就代表着相邻的国家数目。

如果说上一个方法是根据当前搜索状态判断哪个点最紧张。现在这个方法就是判断哪个点的潜在限制更大。

例如：点 A 剩余可使用颜色为 2，度数为 3；点 B 剩余可使用颜色为 2，度数为 6。那么对比起先涂 A 点，我们会先优先选择风险更高的点 B。这样失败暴露得更早，不会等到后面搜索很深之后才发现。

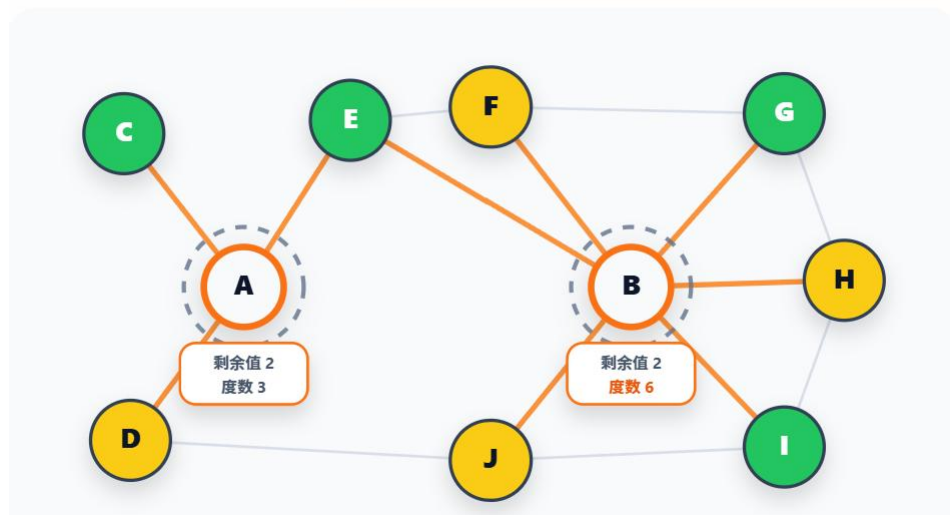


图 2-3

优化颜色的顺序:

除了对选点的顺序做调整，我们还可以优先选择对邻接点影响最小的颜色，让后面的点尽量保留更多选择。

什么意思呢？例如：

我们有一个待涂点 v 可选颜色：红、绿、蓝

V 的邻接点是 A 、 B 和 C

A 可选颜色：红、绿

B 可选颜色：绿、蓝

C 可选颜色：红

如果 v 选择红色，那么 A 、 B 和 C 分别剩余：1、2、0 个可涂色数量。

如果 v 选择绿色，那么 A 、 B 和 C 分别剩余：1、1、1 个可涂色数量。

如果 v 选择蓝色，那么 A 、 B 和 C 分别剩余：2、1、1 个可涂色数量。

我们可以设置某个数值 x ，统计选择某种颜色对多少个邻接点没有影响，选择 x 最大的那个颜色。这个例子中，我们可以选蓝色。

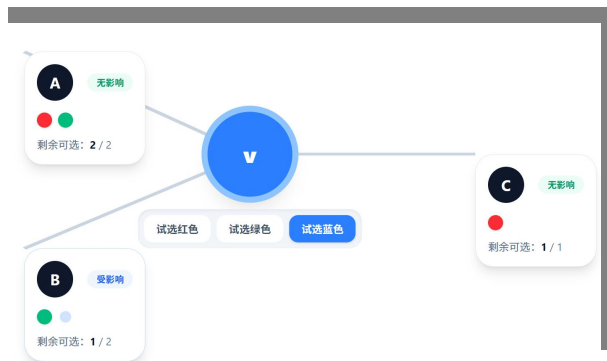


图 2-4

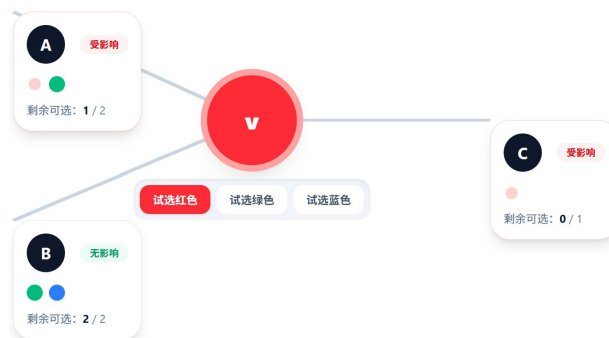


图 2-5

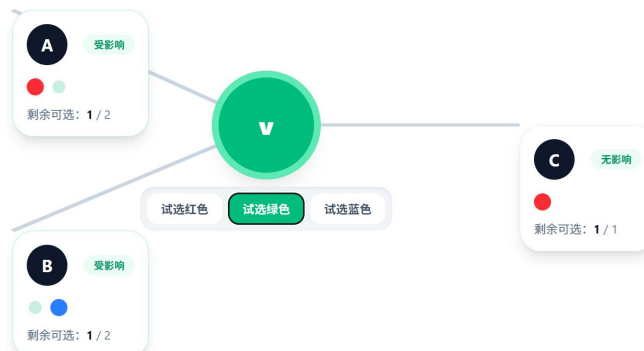


图 2-6

数据的存储形式的优化：

在之前的实验里，我以为比较熟悉邻接矩阵，所有都采用了邻接矩阵的方法存放数

据。

但后来发现邻接矩阵在这个题目中，既不省时间也不省空间。尤其是我们绞尽脑汁想了那么多优化，如果不改变存储方式也只能带来很小的进步。

例如我们进行剩余可填色数最少的点优先的优化方法时。会有一个 for 循环找到可填色数量最少的点，找到以后，还要再次进行一个 for 循环寻找他的邻接区域。如果是邻接表，我们直接就可以找到每个点的邻接区域了。

	A	B	C	D	E	F	
A	x	1	0	0	0	1	$A=[B, F]$
B	1	x	0	1	0	0	$B=[A, D]$
C	0	0	x	0	1	0	$C=[E]$
D	0	1	0	x	0	0	$E=[C]$
E	0	0	1	0	x	0	$F=[A]$
F	1	0	0	0	0	x	
							$\Rightarrow$
邻接矩阵							邻接表

图 2-7

颜色对称性优化原理

在图着色问题中，颜色的编号本身没有实际意义。例如，对于一个 5 色染色方案，如果把所有颜色 1 改成颜色 2，把颜色 2 改成颜色 1，而其他颜色保持对应替换，那么得到的新方案仍然是合法方案。这个新方案和原方案在结构上并没有本质区别，只是颜色名字发生了变化。

因此，如果直接搜索所有方案，就会产生大量由于颜色排列不同而造成的重复搜索。对于 M 种颜色来说，同一类染色方案最多可以通过颜色重命名产生 $M!$ 个方案。也就是说，如果不处理颜色对称性，程序会重复搜索大量本质相同的方案。

为了减少这种重复，本实验采用固定点颜色的方法。所谓固定点，就是在搜索开始前，提前指定某些点必须使用某些颜色。这样做的目的不是改变原问题，而是只搜索其中一种具有代表性的颜色排列，最后再根据颜色排列数还原总方案数。

例如，对于 5 色问题，如果程序固定某个点使用颜色 1，那么原本需要分别考虑这个点使用颜色 1、2、3、4、5 的情况，现在只需要搜索其中一种情况即可。搜索完成后，再乘以 5，就可以得到颜色编号不同情况下的总方案数。

实验结果

平面图形的检查：

首先，可以利用欧拉公式推出的边数限制进行初步筛选。对于顶点数为 V 、边数为 E 的简单平面图，当 $V \geq 3$ 时必须满足 $E \leq 3V - 6$ 。若该条件不满足，则该图一定不是平面图。

但该条件只是必要条件，不能用于充分证明图一定平面。为了得到严格判断，可以进一步使用平面图判定算法。

Kuratowski 定理说明了判断平面图的一个核心标准：一个图是平面图，当且仅当它不包含 K_5 或 $K_{3,3}$ 的细分图。其中 K_5 是 5 个顶点两两相连的完全图， $K_{3,3}$ 是两组三个顶点之间完全相连的二分图，这两类结构都无法在平面上画成无交叉形式。所谓“细分

图”，就是在原来的边上插入一些新的点，但不改变整体连接关系。这个定理虽然能判断出平面图，但对于程序来说，有点过于复杂了。我们不如直接调用已有的库。

例如 Hopcroft-Tarjan 平面性判定算法。在程序实现中，可以调用 NetworkX 库中的 `check_planarity()` 函数，对输入的邻接表或 `.col` 文件进行平面性验证

实验结果：

表 1 单解实验结果

优化方式	花费时间（单位：ms）		
	5 色	15 色	25 色
无优化	> 12h	？（提前中断）	？
最少颜色优化	745.928	？（提前中断）	2.089
最少颜色优化+最大度优化	354.456	？（提前中断）	1.238
最少颜色优化+最大度优化+颜色选择优化+邻接表	287.232	981.239	5.732

表中打问号的点是不能在短时间内得到解的点，若跑了很久还是跑不出来，我将其提前中断了。

可以观察到，对于五色的数据，整体效果比较符合预期。结果基本上都能跑出来，并且随着优化变多，时间都是越来越快。而且最后能跑出来 287ms 左右的成绩，相比起没有优化而言，已经是一个很不错的结果了

对于第二组解，发现普通的优化算法难以跑通，但在最优优化的情况下在选点上进行了一些随机，在多次重复运行中终于顺利得到了解。

对于第三组解，发现整体运行比较顺利。

可将实验结果用 python 代码画出：

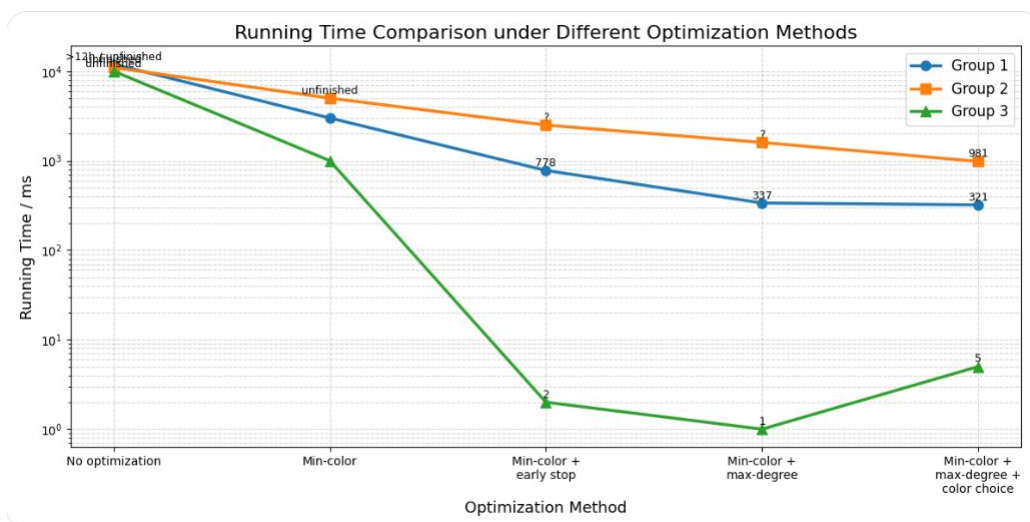


图 3-1

多解的探究

本组实验不再只寻找一个合法染色方案，而是枚举所有合法染色方案。实验继续采用前文的邻接表、最少剩余值优先、度数大的点优先和颜色选择优化，并进一步加入颜色对称性剪枝。

颜色编号本身没有实际意义，因此可以固定部分颜色来减少重复搜索。按点优化是固定一个最高度顶点颜色为 1，最后结果乘以颜色数 m ；按边优化是选择一条真实存在

的边 (u,v)，固定 $u=1, v=2$ ，最后结果乘以 $m \times (m-1)$ 。由于边两端颜色必然不同，按边方式比单点固定消除了更多颜色排列重复。

表 2：多解的结果

数据集	方法	代表解数	还原后总解数	时间	状态	轮数
5a	按点 + 颜色 对称	768	3840	7.598s	完成	154
5a	按边 + 边端 点颜色对称	192	3840	1.916s	完成	39
15b	按点 + 单点 颜色对称	0	0	10.004s	未完成	0
15b	按边 + 边端 点颜色对称	0	0	5.024s	未完成	0
25a	按点 + 单点 颜色对称	65,144, 189	1,628,604,725	5.000s	未完成	13,028,838
25a	按边 + 边端 点颜色对称	68,629,874	41,177,924,400	5.001s	未完成	13,725,975

从结果可以看出，5 色数据能够完整枚举所有解。按点颜色对称需要枚举 768 个代表解，而按边颜色对称只需要枚举 192 个代表解，二者还原后的总解数均为 3840，说明颜色对称性还原是正确的。同时，按边方式运行时间从 7.598s 降低到 1.916s，优化效果明显。

15 色数据在限时内没有完成完整枚举，说明虽然颜色数增多后更容易找到一个可行解，但要枚举所有解时，搜索树反而会急剧膨胀。25 色数据更加明显，5 秒内已经枚举出数千万级代表解，还原后的部分解数达到十亿甚至百亿级，说明多解枚举的难度主要来自解空间规模，而不是单个解是否容易找到。

因此，本实验可以得出结论：颜色对称性优化不会改变问题本身的解空间，只是减少颜色重命名造成的重复枚举；按点固定能减少约 m 倍重复，按边固定能减少约 $m \times (m-1)$ 倍重复。在所有解枚举问题中，按边颜色对称通常比按点颜色对称更有效。

随机生成数据进行实验

并且采用上一小节中“最少颜色优化+最大度优化+颜色选择优化”进行多轮优化后的算法。由于关于算法的优化实验已经在上一节讨论完成，我们将主要聚焦于图与颜色规模的探究

(一)生成不同大小的图

本组实验全都选用 4 种颜色。

表 3

地图区域数	450	600	750	900	1050
时间 (ms)	5.001	7.728	10.029	11.629	12.231

可以直观的看出，当其他变量不变时，时间会随着地区区域数的增长而增长。这个结果也是显然的、意料之中的。

(二)使用不同数量的颜色

我们可以选小规模一点的数据，这样既能够跑出实验结果，也能大致探究颜色数量与时间的关系。

一下实验都采用 20 个区域数目

表 4

颜色数目	4 色	5 色	6 色	7 色
时间花费 (单位: ms)	382.22	107880.291	...	...
方案数	40310784	58195968	...	...

实验总结:

表 5

颜色数	预期剪枝效果	找到一个解	枚举所有解
4 色	强, 容易出现冲突	快	很慢, 但相对最可控
5 色	中等	很快	极慢
6 色	弱	很快	几乎不可行
7 色	很弱	几乎立即	完全不可行

可以总结出，颜色少时我们还能尽可能的去枚举出所有的可能性。但颜色多起来就几乎不可控，可行方案数呈指数增长，很可怕。

四、实验结果及分析：

本次实验围绕“平面图与四色定理”展开，主要完成了平面图概念理解、四色定理验证、回溯法求解地图填色问题，以及多种优化策略对算法性能影响的实验分析。

首先，通过对平面图的学习，我明确了平面图的核心特征：图中的边除了公共端点之外不能相交。同时也纠正了自己原先的一个误区，即图中的顶点必须是事先规定好的，不能因为两条边在绘图时相交，就临时把交点当作新的顶点。这个认识对后续将地图抽象成图结构具有重要意义。

其次，在小规模地图数据实验中，我将地图区域抽象为图中的顶点，将相邻区域抽象为边，并通过 `.col` 文件存储图结构。利用回溯法对图进行四色填色，最终成功得到一种合法的染色方案，并通过程序验证了相邻区域颜色均不相同，说明该小规模平面图满足四色定理。该实验帮助我更加直观地理解了四色定理在地图填色问题中的应用。

在算法实现方面，最基础的回溯法通过逐个节点尝试颜色，并在出现冲突时回退到上一步重新选择颜色。该方法逻辑清晰、容易实现，但在数据规模增大时会产生大量无效搜索，运行效率较低。因此，本实验进一步引入了多种优化策略。

其中，“最少剩余值优先”能够优先选择当前可用颜色数量最少的点进行染色，使冲突更早暴露，从而减少不必要的递归搜索；“最少剩余值优先 + 度数大的点优先”则在剩余颜色数量相同的情况下，优先处理相邻区域更多、限制更强的点，进一步提高剪枝效率；“颜色选择优化”则从颜色顺序入手，优先选择对邻接点影响较小的颜色，使后续节点保留更多选择空间。此外，将数据存储方式由邻接矩阵改为邻接表，也显著减少了遍历邻接点时的时间与空间开销。

从实验结果来看，优化策略确实能够明显提升回溯法的效率。对于 5 色数据，随着优化手段的逐步增加，运行时间整体呈下降趋势，从难以在合理时间内完成，到最终能够在约 287ms 内得到结果，说明剪枝和合理的数据结构对搜索算法性能有很大影响。对于 15 色数据，普通优化方法仍然难以跑通，但在综合优化并加入一定随机性的情况下，可以在多次实验中得到解。对于 25 色数据，实验结果反而较快，这说明算法运行时间不仅与颜色数有关，还与具体图结构、搜索顺序以及解空间分布有关。

在随机生成数据实验中，可以观察到当颜色数固定为 4 时，随着地图区域数量从 450 增加到 1050，运行时间整体上升。这说明在其他条件基本一致的情况下，图规模越大，算法需要处理的节点和约束越多，运行时间也会增加。另一方面，在固定区域数为 20 时，随着颜色数量增加，可行方案数迅速增长。虽然找到一个可行解可能会变得更容易，但若要求枚举所有解，搜索空间会随颜色数量呈指数级增长，导致运行时间急剧增加，甚至变得几乎不可行。

通过本次实验，我对地图填色问题、四色定理以及回溯搜索算法有了更深入的理解。实验表明，回溯法虽然是一种通用且直观的求解方法，但其效率高度依赖于剪枝策略、选点顺序、颜色选择顺序以及图的存储结构。合理的优化能够大幅减少搜索空间，提高程序运行效率。同时，本实验也让我认识到，在组合搜索问题中，问题规模稍微增大，解空间就可能出现指数级膨胀，因此算法设计不仅要追求正确性，也要重视效率和可扩展性。总体来说，

本次实验较好地将图论理论、算法设计和程序实现结合起来，加深了我对图着色问题和回溯优化思想的理解。

五、实验结论：

本次实验验证了四色定理在地图填色问题中的应用，即平面图中的相邻区域最多可用四种颜色完成合法染色。通过回溯法可以正确求解地图填色问题，但原始回溯法在数据规模较大时效率较低。

实验结果表明，采用最少剩余值优先、度数大的点优先、颜色选择优化以及邻接表存储后，算法运行效率明显提升，搜索过程中的无效分支大幅减少。同时，图的规模越大，运行时间总体越长；颜色数量增加时，若只寻找一个解可能较快，但若枚举所有解，方案数会迅速增长，计算难度显著增加。

总体来看，合理的剪枝策略和数据结构优化能够有效提高回溯法求解图着色问题的性能。



指导教师批阅意见：	
成绩评定：	
指导教师签字：	
月 日	2026 年
备注：	

注：1、报告内的项目或内容设置，可根据实际情况加以调整和补充。
2、教师批改学生实验报告时间应在学生提交实验报告时间后 10 日内。